Final Report

Exercise 1: Real-Time Sound Classification

Real-Time Sound Classification Using Microphone Input, FFT Feature Extraction, OSC Communication, and KNN Classification in Wekinator

Processing and Wekinator

Exercise 1 Report

Contents

1 Title . . . 4
2 Abstract . . . 4
3 Objective of the Exercise . . . 4
4 Tools and Libraries Used . . . 4
4.1 Processing . . . 4
4.2 Processing Sound Library . . . 5
4.3 oscP5 Library . . . 5
4.4 Wekinator . . . 5
5 Complete System Architecture . . . 5
6 Input Processing Sketch . . . 6
6.1 Purpose of the Input Sketch . . . 6
6.2 Audio Input . . . 6
6.3 FFT Analysis . . . 6
6.4 Meaning of FFT . . . 7
6.5 Feature Modes . . . 7
6.6 Downsampling from 512 FFT Bands to 12 Bands . . . 8
6.7 Feature Vector Sent to Wekinator . . . 8
6.8 OSC Configuration of the Input Sketch . . . 8
6.9 Sending Frequency . . . 9
7 Wekinator Project . . . 9
7.1 Role of Wekinator . . . 9
7.2 Wekinator Input Settings . . . 9
7.3 Wekinator Output Settings . . . 9
7.4 Why the Project Uses 1 Output and 2 Classes . . . 10
7.5 Class Definitions . . . 10
8 KNN Classifier . . . 10
8.1 What KNN Is . . . 10
8.2 KNN with k = 3 . . . 11
8.3 Why k = 3 Is Better Than k = 1 . . . 11
9 Training Data . . . 11
10 Training Process . . . 11
10.1 Step 1 — Run the Input Sketch . . . 11
10.2 Step 2 — Create or Open the Wekinator Project . . . 12
10.3 Step 3 — Record Class 1 . . . 12
10.4 Step 4 — Record Class 2 . . . 12
10.5 Step 5 — Repeat Recordings . . . 12
10.6 Step 6 — Set KNN to k = 3 . . . 13
10.7 Step 7 — Train the Model . . . 13
10.8 Step 8 — Run the Model . . . 13
11 Output Processing Sketch . . . 13
11.1 Purpose of the Output Sketch . . . 13
11.2 OSC Receiving Settings . . . 14
11.3 Class-to-Color Mapping . . . 14
11.4 Why the Output Sketch Is Separate . . . 14
12 Correct Running Order . . . 14
12.1 During Training . . . 14
12.2 During Testing and Demonstration . . . 15
13 Final Tested Configuration . . . 15

14 Results . . . 16
15 Discussion . . . 16
15.1 Importance of Feature Extraction . . . 16
15.2 Feature Dimensionality . . . 16
15.3 Full FFT with 512 Bands . . . 17
15.4 Reduced FFT with 64 Bands . . . 17
15.5 Reduced FFT with 12 Bands . . . 17
15.6 Mean and Variance . . . 18
15.7 Spectral Features vs Statistical Features . . . 18
16 Common Problems and Debugging . . . 18
16.1 Wekinator Does Not Receive Inputs . . . 18
16.2 Wekinator Predicts the Wrong Class . . . 19
16.3 Output Sketch Does Not Change Color . . . 19
17 What Was Learned . . . 19
17.1 Real-Time Audio Processing . . . 19
17.2 FFT as Feature Extraction . . . 19
17.3 Machine Learning Workflow . . . 19
17.4 OSC Communication . . . 19
17.5 Interactive System Design . . . 20
18 Deliverables . . . 20
18.1 Input Processing Project . . . 20
18.2 Output Processing Project . . . 20
18.3 Wekinator Project . . . 20
18.4 Report . . . 20
19 Final Conclusion . . . 20

1. Title

Real-Time Sound Classification Using Microphone Input, FFT Feature Extraction, OSC Communication, and KNN Classification in Wekinator

2. Abstract

This exercise implements a real-time sound classification system using Processing and Wekinator. The system captures live audio from a microphone, extracts numerical audio features using the Fast Fourier Transform (FFT), sends those features to Wekinator through OSC, trains a K-Nearest Neighbors classifier, and visualizes the predicted class in a second Processing sketch.

The implemented and tested configuration uses **12 reduced FFT bands** as input features. Wekinator is configured with **12 inputs**, **1 classifier output**, **2 classes**, and a **KNN classifier with k = 3**. The two trained sound classes are vocal sounds: **"eeeeeeee"** for Class 1 and **"uuuuuuuu"** for Class 2. The output Processing sketch receives the predicted class from Wekinator and changes the canvas color: red for Class 1 and blue for Class 2.

The project demonstrates the complete workflow of an interactive machine learning system: real-time sensing, feature extraction, machine learning classification, OSC communication, and real-time visual feedback.

3. Objective of the Exercise

The objective of this exercise is to design and develop an interactive system capable of classifying live sounds in real time.

The system must:

1. Capture live audio from a microphone.
2. Analyze the sound using FFT.
3. Extract useful numerical features from the audio signal.
4. Send these features from Processing to Wekinator using OSC.
5. Train a machine learning classifier in Wekinator.
6. Use the trained classifier to recognize two different sound classes.
7. Send the predicted class from Wekinator to a second Processing sketch.
8. Visualize the predicted class by changing the canvas color.

The exercise also aims to show how different feature representations can affect classification performance. These feature representations include:

- Full FFT with 512 bands.
- Reduced FFT with 64 bands.
- Reduced FFT with 12 bands.
- Mean and variance of the 512 FFT bands.

In the current completed version, the system was implemented and tested using the **12-band FFT representation**.

4. Tools and Libraries Used

4.1. Processing

Processing is used to create the two sketches required by the exercise:

1. **Input sketch**: captures microphone audio, extracts features, and sends them to Wekinator.
2. **Output sketch**: receives the predicted class from Wekinator and changes the screen color.

Processing is suitable for this task because it is designed for creative coding, real-time interaction, graphics, sound, and communication with external tools.

4.2. Processing Sound Library

The Processing Sound library is used in the input sketch.

It provides the following important classes:

```
AudioIn
FFT
```

`AudioIn` captures live sound from the microphone.

`FFT` converts the microphone audio signal into frequency-domain information.

4.3. oscP5 Library

The oscP5 library is used for OSC communication.

OSC stands for **Open Sound Control**.

In this project, OSC is used to connect Processing and Wekinator:

```
Processing input sketch → Wekinator
Wekinator → Processing output sketch
```

4.4. Wekinator

Wekinator is used as the machine learning tool.

It receives feature vectors from the input Processing sketch, records examples for each class, trains a classifier, and sends the predicted class to the output Processing sketch.

In this exercise, Wekinator is configured as a **two-class classifier** using **K-Nearest Neighbors**.

5. Complete System Architecture

The complete system contains three connected parts:

```
1. Processing input sketch
2. Wekinator classifier
3. Processing output sketch
```

The complete data flow is:

```
User makes a sound
        ↓
Microphone captures audio
        ↓
Processing input sketch receives audio
        ↓
FFT analysis extracts frequency features
        ↓
Processing sends features to Wekinator using /wek/inputs
        ↓
Wekinator classifies the sound
        ↓
Wekinator sends predicted class using /wek/outputs
        ↓
Processing output sketch receives class
        ↓
Canvas color changes according to class
```

The system therefore creates a complete real-time interaction loop:

```
human sound → audio analysis → machine learning → visual feedback
```

6. Input Processing Sketch

6.1. Purpose of the Input Sketch

The input sketch is responsible for the first half of the system.

Its role is:

```
microphone audio → feature extraction → Wekinator input
```

It does not classify the sound by itself. It only prepares and sends numerical data to Wekinator.

The input sketch performs the following tasks:

1. Starts the microphone.
2. Computes a 512-band FFT from the live audio.
3. Creates the selected feature representation.
4. Sends the feature vector to Wekinator using OSC.
5. Displays a real-time visualization of the features.

6.2. Audio Input

The sketch uses:

```
AudioIn mic;
```

This object represents the microphone input.

The microphone is started with:

```
mic = new AudioIn(this, 0);
mic.start();
```

The number `0` means the first available audio input device.

This allows Processing to receive live sound from the computer microphone.

6.3. FFT Analysis

The sketch uses:

```
FFT fft;
```

The FFT is configured with:

```
int bands = 512;
float[] spectrum = new float[bands];
```

This means the audio is analyzed into 512 frequency bands.

The FFT is connected to the microphone using:

```
fft = new FFT(this, bands);
fft.input(mic);
```

During each frame, the sketch calls:

```
fft.analyze(spectrum);
```

This fills the `spectrum` array with 512 values.

Each value represents the energy of a frequency band.

6.4. Meaning of FFT

FFT means **Fast Fourier Transform**.

A microphone signal is originally a time-domain signal. This means it describes how air pressure changes over time.

However, sound classes are often easier to distinguish by their frequency content.

FFT converts the sound from:

```
time-domain signal
```

into:

```
frequency-domain representation
```

In simple terms, FFT answers the question:

```
Which frequencies are present in this sound, and how strong are they?
```

This is useful for sound classification because different sounds often produce different frequency patterns.

For example, the vocal sound `eeeeeeee` and the vocal sound `uuuuuuuu` have different spectral shapes because the mouth shape and resonance are different.

6.5. Feature Modes

The input sketch supports four feature modes:

Feature mode	Feature representation	Number of Wekinator inputs
1	Full FFT	512
2	Reduced FFT	64
3	Reduced FFT	12
4	Mean + variance	2

The current implemented project uses:

```
int featureMode = 3;
```

This means the sketch sends 12 reduced FFT features to Wekinator.

6.6. Downsampling from 512 FFT Bands to 12 Bands

The FFT produces 512 frequency values. In the tested version, these 512 values are reduced to 12 values.

This process is called **downsampling**.

The idea is:

```
512 detailed frequency bands
        ↓
group neighboring bands
        ↓
calculate the average of each group
        ↓
12 broad frequency features
```

This reduces the input size while keeping the general shape of the spectrum.

The advantage of using 12 features is that the Wekinator project becomes easier to configure, faster to train, and easier to debug.

The disadvantage is that some detailed frequency information is lost.

6.7. Feature Vector Sent to Wekinator

For the current project, the input sketch sends a vector of 12 numbers.

Conceptually, one message looks like this:

```
/wek/inputs f1 f2 f3 f4 f5 f6 f7 f8 f9 f10 f11 f12
```

Each `f` value is one reduced FFT feature.

These 12 values describe the general frequency shape of the current microphone sound.

6.8. OSC Configuration of the Input Sketch

The input sketch sends data to Wekinator using:

```
OSC message address: /wek/inputs
Destination host: 127.0.0.1
Destination port: 6448
```

This means:

```
Processing sends feature data to Wekinator on the same computer.
```

The important code idea is:

```
OscMessage msg = new OscMessage("/wek/inputs");
```

This creates the OSC message.

Then the features are added to the message:

```
msg.add(features[i]);
```

Finally, the message is sent to Wekinator:

```
oscP5.send(msg, wekinator);
```

6.9. Sending Frequency

The input sketch sends features approximately every 30 milliseconds.

This means about:

```
33 messages per second
```

This is fast enough for real-time interaction.

The system can respond quickly when the user changes from one sound to another.

7. Wekinator Project

7.1. Role of Wekinator

Wekinator is the machine learning part of the project.

It receives the feature vectors from the Processing input sketch and learns how those features correspond to different classes.

In this exercise, Wekinator learns two classes:

```
Class 1 = eeeeeeeeeeeee
Class 2 = uuuuuuuuuuuuu
```

After training, Wekinator receives live feature vectors and predicts whether the current sound is closer to Class 1 or Class 2.

7.2. Wekinator Input Settings

For the current 12-band experiment, Wekinator is configured with:

```
Input port: 6448
Input OSC message: /wek/inputs
Number of inputs: 12
```

These settings must match the Processing input sketch.

The Processing sketch sends 12 features.

Therefore, Wekinator must be set to 12 inputs.

If Wekinator expects a different number of inputs, the project will not work correctly.

7.3. Wekinator Output Settings

Wekinator sends the classification result to the output Processing sketch using:

```
Output OSC message: /wek/outputs
Output host: localhost
Output port: 12000
Number of outputs: 1
Output type: Classification
Number of classes: 2
```

The output host is `localhost`, meaning the same computer.

The output port is `12000`, which is the port where the output Processing sketch listens.

7.4. Why the Project Uses 1 Output and 2 Classes

This is an important concept.

The project has two classes, but only one output.

This is correct because Wekinator sends one number that represents the predicted class.

For example:

```
1.0 = Class 1
2.0 = Class 2
```

So the output is one value, but that one value can represent different class labels.

Therefore, the correct setup is:

```
Number of outputs: 1
Number of classes: 2
```

7.5. Class Definitions

The two sound classes are:

```
Class 1: eeeeeeeeeeeee
Class 2: uuuuuuuuuuuuu
```

During training:

- The class dropdown was set to `1` while recording `eeeeeeeeee` examples.
- The class dropdown was set to `2` while recording `uuuuuuuuuu` examples.

After training:

- Saying `eeeeeeeeee` should produce Class 1.
- Saying `uuuuuuuuuu` should produce Class 2.

8. KNN Classifier

8.1. What KNN Is

KNN means **K-Nearest Neighbors**.

It is a classification algorithm based on similarity.

When a new feature vector arrives, KNN compares it with the examples recorded during training.

It looks for the closest examples and uses them to decide the class.

8.2. KNN with k = 3

In this project, KNN is configured with:

```
k = 3
```

This means the classifier looks at the 3 nearest recorded examples.

Then it chooses the class that appears most often among those 3 examples.

Example:

```
Nearest example 1 = Class 1
Nearest example 2 = Class 1
Nearest example 3 = Class 2
```

The prediction is:

```
Class 1
```

because Class 1 appears twice.

8.3. Why k = 3 Is Better Than k = 1

Using `k = 1` means the prediction depends on only one nearest example.

This can be sensitive to noise or unusual training examples.

Using `k = 3` makes the decision more stable because it considers three examples instead of one.

This reduces the chance that one bad or noisy example controls the prediction.

The exercise requires KNN with `k = 3`, so the Wekinator model was edited and retrained using this value.

9. Training Data

The final Wekinator project contains:

```
Total training examples: 1081
Class 1 examples: 560
Class 2 examples: 521
```

This is a balanced dataset because both classes have a similar number of examples.

A balanced dataset is important because if one class has many more examples than the other, the classifier may become biased toward the larger class.

Here the difference is small:

```
560 examples for Class 1
521 examples for Class 2
```

So the training data is suitable for this experiment.

10. Training Process

10.1. Step 1 — Run the Input Sketch

The first step is to run the input Processing sketch.

This sketch captures microphone audio and sends feature vectors to Wekinator.

During training, the input sketch must stay running.

Without the input sketch, Wekinator does not receive any sound features.

10.2. Step 2 — Create or Open the Wekinator Project

The Wekinator project is configured with:

```
12 inputs
1 output
2 classes
Classifier output type
```

The input and output OSC settings are:

```
Input port: 6448
Input message: /wek/inputs
Output port: 12000
Output message: /wek/outputs
```

10.3. Step 3 — Record Class 1

The class dropdown is set to:

```
1
```

Then the user says:

```
eeeeeeeeeeeee
```

while pressing **Start Recording**.

After enough examples are recorded, **Stop Recording** is pressed.

These examples are stored as Class 1.

10.4. Step 4 — Record Class 2

The class dropdown is changed to:

```
2
```

Then the user says:

```
uuuuuuuuuuuuu
```

while pressing **Start Recording**.

After enough examples are recorded, **Stop Recording** is pressed.

These examples are stored as Class 2.

10.5. Step 5 — Repeat Recordings

Several recordings are made for each class.

This is important because the same sound is not exactly identical every time.

Small changes can happen in:

- Loudness.
- Pitch.
- Mouth position.
- Microphone distance.
- Background noise.

By recording many examples, Wekinator learns a more general pattern for each class.

10.6. Step 6 — Set KNN to k = 3

The model settings are opened in Wekinator.

The model type is K-Nearest Neighbor.

The number of neighbors is set to:

```
3
```

After changing this setting, the model must be trained again.

The old recordings do not need to be deleted.

Only the model needs to be retrained using the existing examples.

10.7. Step 7 — Train the Model

After recording examples and setting `k = 3`, the **Train** button is pressed.

Wekinator builds the classifier from the recorded training data.

After training finishes, Wekinator is ready to classify live sound.

10.8. Step 8 — Run the Model

After training, the **Run** button is pressed.

Now Wekinator listens to the live features from the input sketch and predicts the sound class in real time.

At this stage, **Start Recording** is no longer used.

The correct button after training is **Run**, not **Start Recording**.

11. Output Processing Sketch

11.1. Purpose of the Output Sketch

The output sketch is responsible for showing the result of the classification.

It does not analyze sound.

It does not compute FFT.

It does not train the model.

Its only job is:

```
Receive Wekinator output → change background color
```

11.2. OSC Receiving Settings

The output sketch listens on port:

```
12000
```

using code similar to:

```
oscP5 = new OscP5(this, 12000);
```

It expects Wekinator to send messages with this address:

```
/wek/outputs
```

This matches the Wekinator output configuration.

11.3. Class-to-Color Mapping

The output sketch uses this mapping:

```
Class 1 → red
Class 2 → blue
Unknown / waiting → gray
```

So, for the current trained classes:

```
eeeeeeeeeeeee → Class 1 → red
uuuuuuuuuuuuu → Class 2 → blue
```

This simple color mapping makes the classifier result easy to understand.

11.4. Why the Output Sketch Is Separate

The project uses two Processing sketches because each sketch has a different responsibility.

The input sketch handles:

```
microphone input and feature extraction
```

The output sketch handles:

```
visual feedback from Wekinator prediction
```

This separation makes the project easier to understand and debug.

It also follows the Wekinator workflow, where one program sends inputs and another program receives outputs.

12. Correct Running Order

12.1. During Training

During training, only these parts need to run:

```
1. Input Processing sketch
2. Wekinator
```

The output sketch is not necessary during training because no final visualization is needed yet.

The purpose of training is only to record examples and build the classifier.

12.2. During Testing and Demonstration

During testing, all three parts should run:

```
1. Input Processing sketch
2. Wekinator in Run mode
3. Output Processing sketch
```

The full testing flow is:

```
User says eeeeeeeeeeeee
        ↓
Input sketch sends 12 FFT features
        ↓
Wekinator predicts Class 1
        ↓
Output sketch receives /wek/outputs 1.0
        ↓
Canvas becomes red
```

and:

```
User says uuuuuuuuuuuuu
        ↓
Input sketch sends 12 FFT features
        ↓
Wekinator predicts Class 2
        ↓
Output sketch receives /wek/outputs 2.0
        ↓
Canvas becomes blue
```

13. Final Tested Configuration

The final tested configuration is:

```
Input sketch feature mode: 3
Feature representation: 12 reduced FFT bands
Processing input OSC message: /wek/inputs
Processing input destination port: 6448
Wekinator input count: 12
Wekinator output count: 1
Wekinator output type: classification
Number of classes: 2
Classifier: K-Nearest Neighbors
k = 3
Wekinator output OSC message: /wek/outputs
Wekinator output destination port: 12000
Output sketch visualization: color-only
Class 1 color: red
Class 2 color: blue
```

The trained classes are:

```
Class 1 = eeeeeeeeeeeee
Class 2 = uuuuuuuuuuuuu
```

The training data is:

```
Total examples = 1081
Class 1 examples = 560
Class 2 examples = 521
```

14. Results

The system was successfully trained and tested with two vocal sounds.

After training, Wekinator was switched to Run mode.

When the user said:

```
eeeeeeeeeeeee
```

Wekinator predicted:

```
Class 1
```

and the output Processing sketch displayed a red canvas.

When the user said:

```
uuuuuuuuuuuuu
```

Wekinator predicted:

```
Class 2
```

and the output Processing sketch displayed a blue canvas.

This confirms that the communication chain works correctly:

```
Processing input sketch → Wekinator → Processing output sketch
```

It also confirms that the trained KNN classifier can distinguish the two selected vocal classes using 12 reduced FFT features.

15. Discussion

15.1. Importance of Feature Extraction

The classifier does not receive raw audio directly.

Instead, it receives numerical features extracted from audio.

This is important because machine learning models need structured numerical data.

FFT features are useful because they describe the frequency content of the sound.

The vocal sounds `eeeeeeee` and `uuuuuuuu` are different mainly because their spectral shapes are different. FFT captures this difference better than raw loudness alone.

15.2. Feature Dimensionality

The exercise compares different numbers of features:

```
512 features
64 features
12 features
2 features
```

This is called feature dimensionality.

A larger feature vector can contain more information, but it can also make training more complex.

A smaller feature vector is easier to train and faster to process, but it may lose important details.

15.3. Full FFT with 512 Bands

The 512-band version gives the classifier the most detailed frequency information.

Advantages:

- High spectral detail.
- Can represent subtle differences between sounds.
- Useful for complex sound classes.

Disadvantages:

- High number of inputs.
- More difficult to train.
- May require more examples.
- May include unnecessary noise or small details.

15.4. Reduced FFT with 64 Bands

The 64-band version is a compromise between detail and simplicity.

Advantages:

- Keeps much of the spectral shape.
- Easier to train than 512 bands.
- Less noisy than the full spectrum.

Disadvantages:

- Loses some fine frequency details.
- Still more complex than 12 bands.

15.5. Reduced FFT with 12 Bands

The 12-band version was used in the current completed experiment.

Advantages:

- Simple and easy to configure.
- Fast to train.
- Easier to debug.
- Good for clearly different sound classes.
- Lower risk of overfitting small frequency details.

Disadvantages:

- Loses detailed frequency information.
- May not work well for very similar sounds.

In this experiment, 12 bands were sufficient to distinguish `eeeeeeee` from `uuuuuuuu`.

15.6. Mean and Variance

The mean and variance representation uses only two features.

Advantages:

- Very simple.
- Very fast.
- Easy to understand.

Disadvantages:

- Loses most of the spectral shape.
- May not distinguish sounds with similar overall energy.
- Usually less informative than FFT bands.

For vowel sounds, mean and variance may not be enough because vowel differences depend strongly on the distribution of frequency energy, not only the average and spread.

15.7. Spectral Features vs Statistical Features

Spectral features describe the shape of the sound across frequencies.

Examples:

```
512 FFT bands
64 FFT bands
12 FFT bands
```

Statistical features summarize the spectrum into simple values.

Examples:

```
Mean
Variance
```

Spectral features are usually better when the classification depends on frequency patterns.

Statistical features are useful when a simple summary is enough.

In this sound classification task, spectral features are more appropriate because the sounds are vocal and differ in their frequency structure.

16. Common Problems and Debugging

16.1. Wekinator Does Not Receive Inputs

Possible causes:

```
Input sketch is not running.
Wekinator input port is not 6448.
OSC message is not /wek/inputs.
Wekinator input count does not match Processing feature count.
Microphone is not working.
Processing Sound library is missing.
```

For the current project, Processing sends 12 inputs, so Wekinator must be set to 12 inputs.

16.2. Wekinator Predicts the Wrong Class

Possible causes:

```
Too few training examples.
Class recordings are not clear.
Background noise is high.
Microphone distance changed.
KNN k value is incorrect.
The model was not retrained after changing settings.
```

In this project, the KNN value was changed to `k = 3`, and the model was trained again.

16.3. Output Sketch Does Not Change Color

Possible causes:

```
Wekinator is not in Run mode.
Output sketch is not running.
Wekinator output port is not 12000.
Wekinator output message is not /wek/outputs.
Another program is using port 12000.
```

The output sketch should print received classes in the Processing console. This helps confirm whether messages are arriving.

17. What Was Learned

This exercise demonstrates several important ideas.

17.1. Real-Time Audio Processing

The project shows how Processing can capture microphone audio and process it continuously.

17.2. FFT as Feature Extraction

The project shows how FFT converts live audio into frequency features that can be used for classification.

17.3. Machine Learning Workflow

The project shows the workflow:

```
record examples → train model → run model → test prediction
```

17.4. OSC Communication

The project shows how OSC connects different programs in real time.

Processing sends input features to Wekinator, and Wekinator sends predictions back to Processing.

17.5. Interactive System Design

The project demonstrates a complete interactive system where human sound produces immediate visual feedback.

18. Deliverables

The exercise deliverables include:

18.1. Input Processing Project

This project captures audio, extracts FFT features, and sends data to Wekinator.

Main features:

```
Microphone input
FFT with 512 bands
12-band reduced FFT mode
OSC output to /wek/inputs
Port 6448
```

18.2. Output Processing Project

This project receives Wekinator predictions and changes the canvas color.

Main features:

```
OSC input from /wek/outputs
Port 12000
Class 1 = red
Class 2 = blue
```

18.3. Wekinator Project

This project contains the trained classifier.

Main settings:

```
12 inputs
1 output
2 classes
KNN classifier
k = 3
1081 training examples
```

18.4. Report

This final report explains the system, configuration, training procedure, results, and discussion.

19. Final Conclusion

This exercise successfully implements a complete real-time sound classification system.

The Processing input sketch captures microphone audio and extracts FFT-based features. The selected feature configuration for the completed experiment is the 12-band reduced FFT representation. These features are sent to Wekinator using the OSC message `/wek/inputs` on port `6448`.

Wekinator receives 12 input features and uses a K-Nearest Neighbors classifier with `k = 3` to classify the sound into two classes: `eeeeeeee` and `uuuuuuuu`. The model was trained with 1081 total examples, with a balanced distribution between the two classes.

The Wekinator output is sent through OSC using the message `/wek/outputs` on port `12000`. The Processing output sketch receives this prediction and changes the background color: red for Class 1 and blue for Class 2.

The final system works as a real-time interactive pipeline:

```
live microphone sound → FFT features → Wekinator KNN classifier → visual color feedback
```

This project fulfills the main requirements of Exercise 1 and demonstrates the relationship between audio feature extraction, machine learning classification, OSC communication, and real-time multimedia interaction.